IU International University of Applied Sciences

Habit Tracker App

Phase 1 - Conception Document

Course: Object Oriented and Functional Programming with Python (DLBDSOOFPP01)

Student: Faruq Ishola Raji

MatrNo: 4253139

GitHub: https://github.com/rajifaruq2021-hue/oofpp_habits_project

1. Introduction and Aim

The aim of this project is to design and implement a Python-based Habit Tracking application for a client who wants to help their users build and sustain positive daily and weekly habits. The application must allow users to create habits, check them off when completed, track streaks over time, and analyse their progress - all without a graphical user interface. The focus is on solid backend logic, clean Python code, and two core programming style of programmings: **Object-Oriented Programming (OOP)** and **Functional Programming**.

2. Technical Concept

2.1 The Habit Class (Object-Oriented Design)

The central building block of the application is the **Habit** class, defined in *habit.py*. Each object of this class represents one habit a user wants to track. I chose to encode a habit as a class because a habit is a real-world entity with its own data (name, periodicity, creation date, check-off history) and its own behaviour (checking off, calculating streaks). This is a textbook application of OOP grouping data and actions together.

The Habit class contains the following attributes and methods:

Attribute / Method	Type	Purpose
name	str	The name of the habit (e.g. 'Drink Water')
description	str	A short description of what the habit involves
periodicity	str	'daily' or 'weekly' - how often it must be done
creation_date	str	ISO timestamp of when the habit was first created
checkoffs	list	List of ISO timestamps - one per completed event
checkoff()	method	Records one completion event (now or custom time)
get_completed_periods()	method	Returns unique periods (days/weeks) completed

get_current_streak()	method	Counts consecutive periods up to the latest one
get_longest_streak()	method	Finds the all-time best streak across all history
to_dict() / from_dict()	method	Converts habit to/from dict for saving data

2.2 Data Storage (SQLite Database)

Habit data must persist between sessions. I chose **SQLite** (via Python's built-in *sqlite3* module) because it requires no installation, stores everything in a single file (*habits.db*), and is a real relational database - more professional than a JSON file. The database module (*db.py*) manages two tables:

Table	Columns	Purpose
habits	id, name, description, periodicity, creation_date	One row per habit
checkoffs	id, habit_name (FK), timestamp	One row per check-off event

2.3 Analytics Module (Functional Programming)

All analytics are built in *analytics.py* using the functional programming style of programming. Each function is **pure** - it takes a list of Habit objects as input and returns a result, without changing any data. I use Python's built-in higher-order functions: **filter()**, **map()**, and **max()**.

Function	Uses	Returns
get_all_habits()	-	All habits as a list
get_habits_by_periodicity()	filter()	Habits matching 'daily' or 'weekly'
get_longest_streak_all()	map()+max()	Single highest streak across all habits
get_longest_streak_for_habit()	next()	Longest streak for one named habit
get_habit_summary()	map()	List of summary dicts per habit
get_struggling_habits()	filter()	Habits with checkoffs but streak = 0

2.4 User Interaction (CLI Menu)

Users interact with the application through a numbered command-line menu defined in *main.py*. On launch the app automatically sets up the database and loads 5 pre-defined habits (with 4 weeks of example data). The menu loop stays active until the user chooses to exit. The general user flow is:

* 1 User runs: python main.py
* 2 App sets up database + loads pre-defined habits
* 3 Main menu is displayed
* 4 User picks an option (1-6) and presses Enter

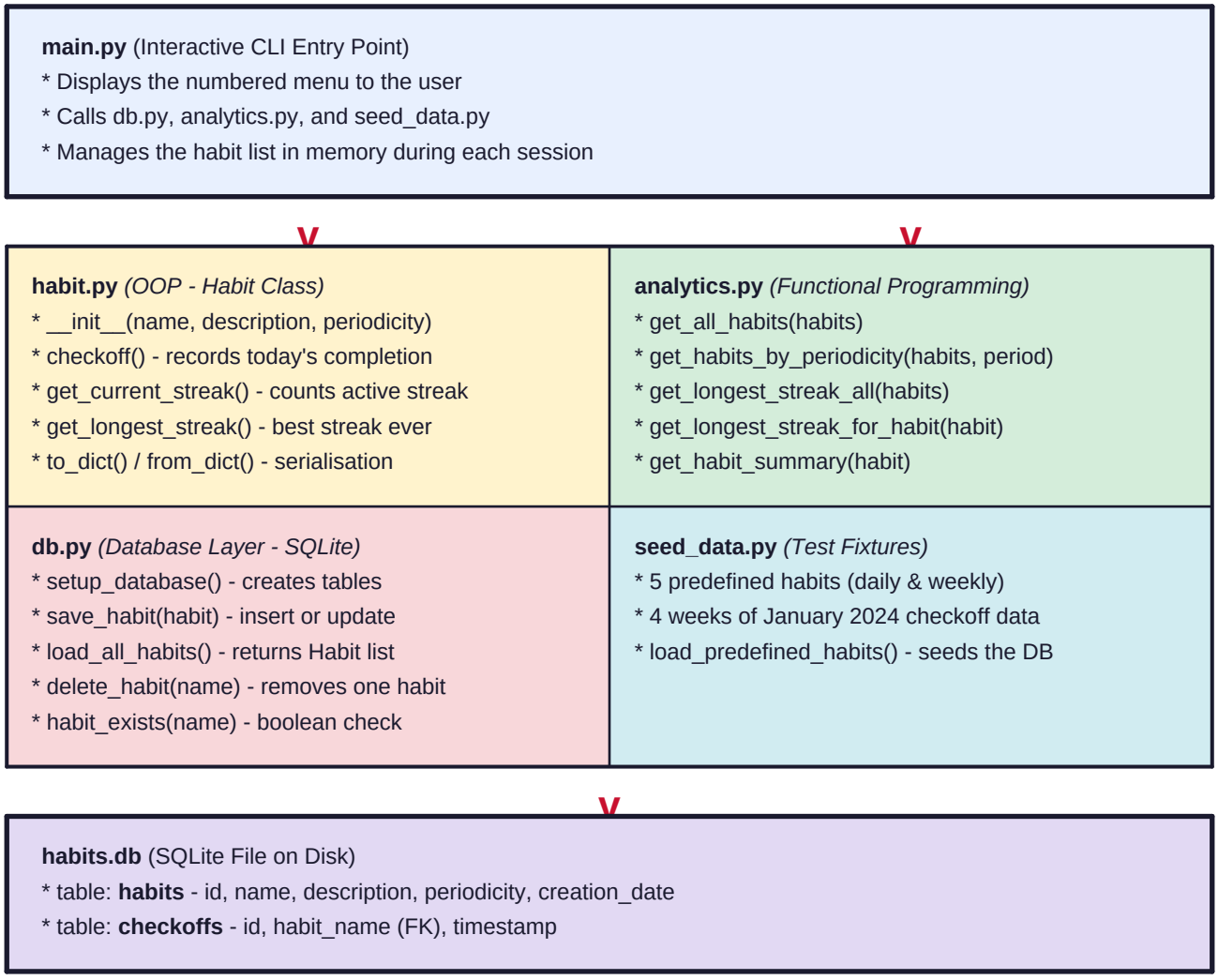
- Option 1: View all habits and streaks
- Option 2: Add a new habit (name, description, periodicity)
- Option 3: Check off a habit (marks it done right now)

- Option 4: Delete a habit (with confirmation)
- Option 5: Analytics sub-menu (5 analysis options)
- Option 6: Exit the application

* 5 Changes are saved to habits.db immediately
* 6 User data persists for the next session

3. Component Interaction Diagram (UML Style)

The diagram below shows how the five modules of the application connect to each other and how data flows between them.



4. Tools and Libraries

Tool / Library	Purpose
Python 3.12	Main programming language
sqlite3 (built-in)	Database - storing and loading habit data

datetime (built-in)	Working with dates, times, and ISO weeks
os (built-in)	Cross-platform terminal clearing
unittest (built-in)	Writing the unit test suite
pytest	Running the test suite from the command line